

**RAJALAKSHMI ENGINEERING COLLEGE**

*Approved by AICTE | Affiliated to Anna University | Accredited by NAAC*

# LABORATORY RECORD NOTEBOOK

Name : DEVANESH S M

Register No : 2116231801029

Year / Semester : 4th / 7

Batch : 2023 - 2027

Branch : B.Tech., Artificial Intelligence and Data Science

Subject Code : CS23633

Subject Name : Cloud Computing

Academic Year : 2026 - 2027

RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

# BONAFIDE CERTIFICATE

Certified that this is a Bonafide Record of the practical work done by **DEVANESH S M**, Register No **2116231801029** of 4th Year / 7th Semester, B.Tech., Artificial Intelligence and Data Science Department in the **CS23633 – Cloud Computing** laboratory during the Academic Year 2026 - 2027.

Faculty in charge

Submitted for the Practical Examination held on: ……

**INTERNAL EXAMINER** **EXTERNAL EXAMINER**

# INDEX


| EX.NO | DATE | NAME OF THE EXPERIMENT | PAGE NO | STAFF SIGN |
|---|---|---|---|---|
| 1 | 24-06-2026 | Private Cloud Setup Using OpenStack | 1 | |
| 2 | 08-07-2026 | Public Cloud Application Deployment Using PaaS | 5 | |
| 3 | 22-07-2026 | Hybrid Cloud Connectivity using AWS Free Tier VPC | 10 | |
| 4 | 05-08-2026 | Basic VPS Server Setup and Network Configuration | 16 | |
| 5 | 07-08-2026 | Cloud Migration and Backup Setup Using Amazon S3 | 22 | |
| 6 | 14-08-2026 | CDN Service Selection and Configuration Using Amazon CloudFront | 29 | |

# EXPERIMENT NO: 1

## SETUP A PRIVATE CLOUD BY PERFORMING THE PROCEDURE USING A SINGLE NODE OPENSTACK IMPLEMENTATION

### AIM

To set up a Private Cloud using a single-node OpenStack implementation and create virtual machine instances.

### OBJECTIVE

- To understand the concept of Private Cloud Computing.
- To install and configure OpenStack using DevStack.
- To create and manage virtual machine instances in a cloud environment.
- To understand cloud resource management using the OpenStack Dashboard.

### SOFTWARE REQUIREMENTS

- Ubuntu Server 22.04 LTS
- OpenStack (DevStack)
- Virtual Machine Software (VirtualBox/VMware)
- Internet Connection
- Web Browser

### THEORY

Cloud Computing is a technology that provides computing resources such as servers, storage, databases, networking, and software over the internet. A Private Cloud is a cloud infrastructure dedicated to a single organization and offers greater control, security, and customization.

OpenStack is an open-source cloud computing platform used for building and managing private and public cloud environments. It provides Infrastructure as a Service (IaaS) through a collection of services for computing, networking, storage, and identity management.

The major services of OpenStack include:

- Nova – Compute Service
- Neutron – Networking Service
- Glance – Image Service
- Keystone – Identity Service
- Cinder – Block Storage Service
- Swift – Object Storage Service

## ARCHITECTURE

OpenStack Private Cloud Architecture:

User → Horizon Dashboard → Keystone Authentication → Nova Compute → Neutron Networking → Glance Images → Cinder Storage

## PROCEDURE

### STEP 1: INSTALL UBUNTU SERVER

1. Create a virtual machine using VirtualBox or VMware.
2. Allocate at least 4 GB RAM and 2 CPU cores.
3. Install Ubuntu Server.

### STEP 2: UPDATE SYSTEM PACKAGES

Execute the following commands:

```
sudo   apt   update
sudo apt upgrade -y
```

### STEP 3: INSTALL GIT

Install Git for downloading DevStack:

```
sudo apt install git -y
```

### STEP 4: DOWNLOAD DEVSTACK

Clone the DevStack repository:

```
git clone https://opendev.org/openstack/devstack
```

### STEP 5: CONFIGURE DEVSTACK

Navigate to the DevStack directory:

```
cd devstack
```

Create a configuration file named local.conf and provide the required credentials.

### STEP 6: INSTALL OPENSTACK

Run the installation script:

```
./stack.sh
```

Wait for the installation process to complete successfully.

**STEP 7: ACCESS OPENSTACK**

**DASHBOARD**

Open a web browser and enter:

```
http://<server-ip>/dashboard
```

Login using the administrator credentials.

**STEP 8: CREATE PROJECT AND USER**

1. Open Identity → Projects.
2. Create a new project.
3. Create users.
4. Assign appropriate roles.

**STEP 9: LAUNCH VIRTUAL MACHINE INSTANCE**

1. Open Compute → Instances.
2. Click Launch Instance.
3. Select image, flavor, and network.
4. Launch the instance.

**STEP 10: VERIFY CONNECTIVITY**

1. Assign a floating IP address.
2. Connect to the instance using SSH.
3. Verify connectivity using ping commands.

Commands Used

```
sudo   apt   update
sudo apt upgrade -y
sudo apt install git -y
git clone https://opendev.org/openstack/devstack
cd devstack
./stack.sh
openstack   server   list
openstack  network  list
openstack image list
```

**OBSERVATIONS**

| Parameter | Status |
|---|---|
| Ubuntu Installation | Successful |
| DevStack Installation | Successful |
| OpenStack Dashboard Access | Successful |
| Project Creation | Successful |
| VM Instance Creation | Successful |
| Network Connectivity | Verified |

**EXPECTED OUTPUT**

- OpenStack Dashboard accessible through a web browser.
- Project, users, and roles successfully created.
- Virtual machine instances launched successfully.
- Network connectivity verified.

**RESULT**

A Private Cloud environment was successfully created using OpenStack. Virtual machine instances were launched and managed through the OpenStack Dashboard.

# EXPERIMENT NO: 2

## DEVELOP A SIMPLE APPLICATION TO UNDERSTAND THE CONCEPT OF PAAS USING GAE/AMAZON ELASTIC BEANSTALK/IBM BLUE MIX/GCC AND LAUNCH IT.

### AIM

To develop and deploy a simple web application using a Platform as a Service (PaaS) cloud environment such as Google App Engine, Amazon Elastic Beanstalk, or IBM Cloud.

### OBJECTIVE

- To understand the concept of Public Cloud Computing.
- To learn the Platform as a Service (PaaS) model.
- To deploy and manage applications in a cloud environment.
- To understand cloud-based application hosting and scaling.

### SOFTWARE REQUIREMENTS

- Google Cloud Platform (GCP) / AWS Account
- Google App Engine or Amazon Elastic Beanstalk
- Python/Java/Node.js Runtime
- Web Browser
- Internet Connection
- Text Editor or IDE

### THEORY

Public Cloud is a cloud deployment model in which computing resources such as servers, storage, databases, and networking are owned and managed by a third-party cloud provider and delivered over the internet. Users can access these resources on demand and pay only for what they use.Platform as a Service (PaaS) is a cloud service model that provides developers with a complete environment for developing, testing, deploying, and managing applications. The cloud provider manages the underlying infrastructure, operating systems, middleware, and runtime environments, allowing developers to focus only on application development.Examples of PaaS platforms include Google App Engine, Amazon Elastic Beanstalk, Microsoft Azure App Service, and IBM Cloud.

### ADVANTAGES OF PAAS

- Faster application development and deployment.
- Reduced infrastructure management.
- Automatic scaling and load balancing.

## ARCHITECTURE

PaaS Architecture:

```
User
  ↓
Web Browser
  ↓
Cloud Platform (PaaS)
  ↓
Runtime Environment
  ↓
Application Instance
  ↓
Cloud Storage / Database
```

## ALGORITHM

1. Create an account on the cloud platform.
2. Develop a simple web application.
3. Configure the application deployment settings.
4. Install and configure Cloud SDK.
5. Create a cloud project.
6. Deploy the application to the cloud platform.
7. Verify application deployment.
8. Access the application through the generated URL.

## FLOWCHART

```
Start
  ↓
Create Cloud Account
  ↓
Develop Application
  ↓
Configure Application
  ↓
Install Cloud SDK
  ↓
Deploy Application
  ↓
Verify Deployment
  ↓
Access Application URL
  ↓
Stop
```

**PROCEDURE**

**STEP 1: CREATE A CLOUD ACCOUNT**

1. Create an account on Google Cloud Platform or AWS.
2. Sign in to the cloud console.
3. Enable billing if required.

**STEP 2: CREATE A SIMPLE APPLICATION**

Create a sample web application.

Example (Python Flask):

```
from flask import Flask

app = Flask(_name_)

@app.route('/')
def home():
    return "Hello Cloud Computing"

if _name__== '_main_':
    app.run()
```

**STEP 3: CREATE APPLICATION CONFIGURATION FILE**

Create a configuration file.

Example (app.yaml):

```
runtime: python39

handlers:
- url: /.*
  script: auto
```

**STEP 4: INSTALL CLOUD SDK**

Install Google Cloud SDK or AWS CLI.

Verify installation:

```
gcloud version
```

**STEP 5: LOGIN TO CLOUD PLATFORM**

```
gcloud auth login
```

**STEP 6: CREATE CLOUD PROJECT**

Create a new project from the Google Cloud Console.

Project Name: cloud-lab-project

Alternatively, a project may be created using the Google Cloud SDK:

```
gcloud projects create cloud-lab-project
```

**STEP 7: SET ACTIVE PROJECT**

Configure the newly created project as the active project:

```
gcloud config set project cloud-lab-project
```

Verify the configuration:

```
gcloud config list
```

**STEP 8: DEPLOY APPLICATION**

Deploy the application:

```
gcloud app deploy
```

**STEP 9: LAUNCH APPLICATION**

```
gcloud app browse
```

**STEP 10: VERIFY DEPLOYMENT**

1. Open the generated URL.
2. Verify that the application loads successfully.
3. Test application functionality.

**COMMANDS USED**

```
gcloud auth login
gcloud projects create cloud-lab-project
gcloud config set project cloud-lab-project
gcloud app create
gcloud app deploy
gcloud app browse
```

**OBSERVATIONS**

| Parameter | Status |
|---|---|
| Cloud Account Creation | Successful |
| Project Creation | Successful |
| Application Development | Successful |
| Application Deployment | Successful |
| URL Generation | Successful |
| Application Access | Verified |

**EXPECTED OUTPUT**

- Cloud project created successfully.
- Application deployed on the cloud platform.
- Public URL generated.
- Application accessible through a web browser.
- Successful execution of application services.

**RESULT**

A simple web application was successfully deployed on a Public Cloud Platform using the PaaS model. The application was accessible through a web browser and managed by the cloud provider.

# EXPERIMENT NO: 3

## Setting Up Hybrid Connectivity Experiment with setting up a secure VPN connection between an on-premises data center and a public cloud provider such as AWS or Azure.

**AIM**

To implement a Hybrid Cloud environment using Amazon Web Services Free Tier by configuring a Virtual Private Cloud (VPC) with subnets and route tables, and establishing secure connectivity with a local on-premises system using OpenVPN.

**OBJECTIVE**

- To create a real VPC in AWS Free Tier.
- To configure subnets and route tables.
- To deploy an EC2 instance in cloud.
- To simulate on-premises system using VirtualBox.
- To establish secure VPN connectivity between cloud and local system.
- To understand Hybrid Cloud architecture practically.

**SOFTWARE REQUIREMENTS**

- AWS Free Tier Account
- VirtualBox
- Ubuntu 20.04 / 22.04 LTS (Local VM + EC2)
- OpenVPN
- Web Browser
- SSH Client (Optional)

**THEORY**

Hybrid Cloud is a computing model that combines on-premises infrastructure with public cloud services.

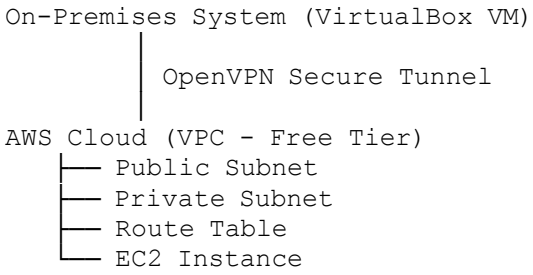
In this experiment:

- AWS provides cloud infrastructure (VPC, EC2, networking)
- Local VirtualBox VM acts as on-premises system
- OpenVPN provides secure encrypted communication

**AWS COMPONENTS:**

- **VPC (Virtual Private Cloud):** Isolated cloud network
- **Subnet:** Divides VPC into smaller networks
- **Route Table:** Controls traffic flow
- **Internet Gateway:** Enables internet access
- **EC2 Instance:** Virtual machine in cloud

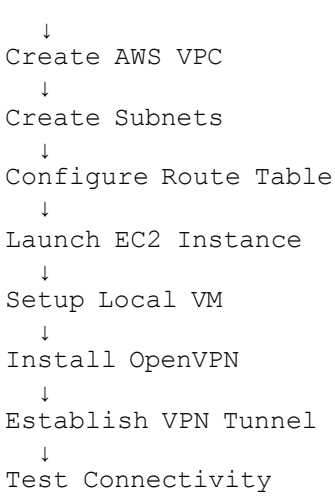
**ARCHITECTURE**




**ALGORITHM**

1. Create AWS Free Tier account.
2. Create VPC with CIDR block.
3. Create public and private subnets.
4. Attach Internet Gateway.
5. Configure route tables.
6. Launch EC2 instance.
7. Setup local VM using VirtualBox.
8. Install OpenVPN on both systems.
9. Configure VPN connection.
10. Verify connectivity using ping.

**FLOWCHART**



**PROCEDURE**

**STEP 1: CREATE VPC IN AWS**

1. Login to AWS Console.
2. Go to VPC Dashboard.
3. Click “Create VPC”.
4. Enter:
    - Name: Hybrid-VPC
    - CIDR: 10.0.0.0/16
5. Create VPC.

**STEP 2: CREATE SUBNETS**

- Public Subnet: 10.0.1.0/24
- Private Subnet: 10.0.2.0/24

Associate both with VPC.

**STEP 3: CREATE INTERNET GATEWAY**

1. Create Internet Gateway (IGW).
2. Attach it to VPC.

**STEP 4: CONFIGURE ROUTE TABLE**

Add route:

| Destination | Target |
|---|---|
| 0.0.0.0/0 | Internet Gateway |

Associate with Public Subnet

**STEP 5: LAUNCH EC2 INSTANCE**

1. Go to EC2 → Launch Instance
2. Choose Ubuntu (Free Tier eligible)
3. Select VPC and Public Subnet
4. Enable SSH (port 22)
5. Launch instance

**STEP 6: SETUP LOCAL VM (ON-PREMISES)**

Using VirtualBox:

- Install Ubuntu Linux
- Assign IP: 192.168.1.10

**STEP 7: INSTALL OPENVPN**

On both AWS EC2 and Local VM:

```
sudo apt update
sudo apt install openvpn -y
```

**STEP 8: CONFIGURE VPN SERVER (AWS EC2)**

```
sudo apt install easy-rsa -y
sudo systemctl start openvpn
sudo systemctl enable openvpn
```

**STEP 9: CONFIGURE VPN CLIENT (LOCAL VM)**

```
sudo openvpn --config client.ovpn
```

**STEP 10: VERIFY CONNECTIVITY**

From Local VM:

```
ping <EC2-private-IP>
```

From AWS EC2:

```
ping 192.168.1.10
```

**COMMANDS USED**

```
sudo apt update

sudo apt install openvpn -y

sudo apt install easy-rsa -y

sudo systemctl start openvpn

sudo systemctl enable openvpn

ping <ip-address>
```

## OBSERVATIONS

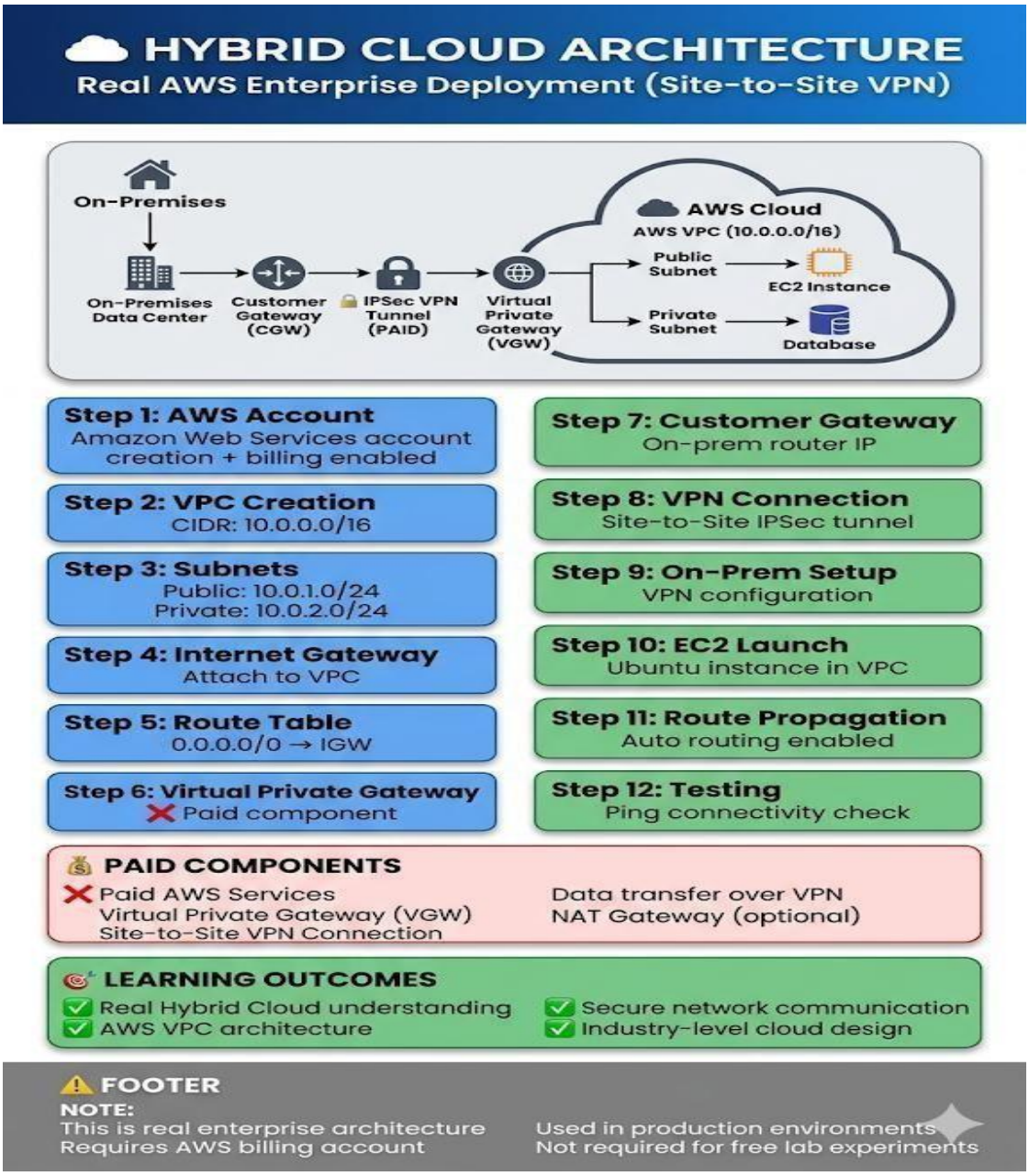
| Parameter | Status |
|---|---|
| VPC Creation | Successful |
| Subnet Configuration | Successful |
| Route Table Setup | Successful |
| EC2 Instance Deployment | Successful |
| VPN Setup | Successful |
| Connectivity Test | Successful |

## EXPECTED OUTPUT

- AWS VPC created successfully
- Public and private subnets configured
- Route tables properly assigned
- EC2 instance running in cloud
- Local VM connected as on-premises system
- Secure VPN tunnel established
- Successful communication between cloud and local system

## RESULT

A Hybrid Cloud environment was successfully implemented using AWS Free Tier VPC and a local VirtualBox system. Secure communication between cloud and on-premises systems was established using OpenVPN.




**Note:** The following steps outline the operational workflow of a fully paid AWS environment.

# EXPERIMENT NO: 4

## PROVISION A VPS INSTANCE FROM A CLOUD PROVIDER OR SET UP A VIRTUAL MACHINE ON YOUR LOCAL SYSTEM USING SOFTWARE LIKE VIRTUALBOX OR VMWARE.

### AIM

To provision a Virtual Private Server (VPS) using VirtualBox, install Ubuntu Server, configure network settings (IP Address, Subnet Mask, Gateway, DNS), and enable SSH remote access.

### OBJECTIVES

1. To understand the concept of Virtual Private Server (VPS).
2. To create and manage virtual machines using VirtualBox.
3. To install Ubuntu Server OS.
4. To configure static network settings.
5. To enable SSH remote access.

### SOFTWARE REQUIREMENTS

- Oracle VirtualBox
- Ubuntu Server 22.04 LTS ISO
- Windows/Linux Host System
- Internet Connection
- Terminal Access

### THEORY

A Virtual Private Server (VPS) is a software-based server created using virtualization technology. It operates independently with its own operating system, storage, memory, and network configuration while sharing the physical hardware with other virtual machines.

Virtualization allows efficient utilization of resources and forms the foundation of cloud computing platforms.

Network configuration parameters include:

### IP ADDRESS

A unique address assigned to a device on a network.

**SUBNET MASK**

Determines the network and host portions of an IP address.

**GATEWAY**

Provides access to networks outside the local network.

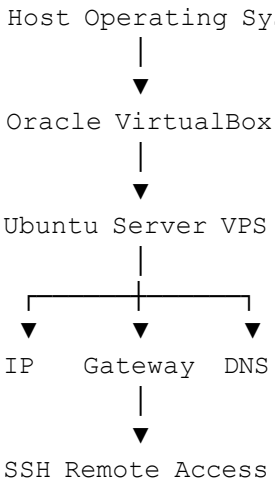
**DNS (DOMAIN NAME SYSTEM)**

Converts domain names into IP addresses.

**SSH (SECURE SHELL)**

A secure protocol used for remote administration of Linux systems.

**ARCHITECTURE**




**ALGORITHM**

1. Install Oracle VirtualBox.
2. Create a new virtual machine.
3. Attach Ubuntu Server ISO image.
4. Install Ubuntu Server.
5. Update system packages.
6. Identify network interface.
7. Configure static IP address.
8. Configure subnet mask.
9. Configure default gateway.
10. Configure DNS server.

**PROCEDURE**

**STEP 1: INSTALL VIRTUALBOX**

Download and install Oracle VirtualBox.

**STEP 2: CREATE VIRTUAL**

**MACHINE**

Configure:

| Parameter | Value |
|---|---|
| Name | VPS-Server |
| OS Type | Ubuntu (64-bit) |
| RAM | 2 GB |
| CPU | 2 Cores |
| Storage | 20 GB |

**STEP 3: INSTALL UBUNTU SERVER**

1. Attach Ubuntu Server ISO.
2. Start VM.
3. Follow installation wizard.
4. Create username and password.
5. Complete installation.

**STEP 4: UPDATE UBUNTU**

```
sudo apt update
sudo apt upgrade -y
```

**STEP 5: IDENTIFY NETWORK INTERFACE**

```
ip a
```

Sample Interface:

```
ens33
```

**STEP 6: CONFIGURE STATIC IP**

Edit Netplan file:

```
sudo nano /etc/netplan/00-installer-config.yaml
```

Add:

```
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: no
      addresses:
        - 192.168.1.100/24
      gateway4: 192.168.1.1
      nameservers:
        addresses:
          - 8.8.8.8
          - 8.8.4.4
```

Apply configuration:

```
sudo netplan apply
```

**STEP 7: VERIFY NETWORK CONFIGURATION**

Check IP:

```
ip addr
```

Check routing table:

```
ip route
```

Check DNS:

```
cat /etc/resolv.conf
```

**STEP 8: TEST CONNECTIVITY**

Test Internet:

```
ping -c 4 8.8.8.8
```

Test DNS:

```
ping -c 4 google.com
```

**STEP 9: INSTALL SSH SERVER**

```
sudo apt install openssh-server -y
```

**STEP 10: ENABLE SSH SERVICE**

```
sudo systemctl enable ssh
sudo systemctl start ssh
```

Verify status:

```
sudo systemctl status ssh
```

**STEP 11: CONNECT REMOTELY**

From another system:

```
ssh username@192.168.1.100
```

**COMMANDS USED**

```
sudo apt update
sudo apt upgrade -y
ip a
sudo nano /etc/netplan/00-installer-config.yaml
sudo netplan apply
ip addr
ip route
cat /etc/resolv.conf
ping -c 4 8.8.8.8
ping -c 4 google.com
sudo apt install openssh-server -y
sudo systemctl enable ssh
sudo systemctl start ssh
sudo systemctl status ssh
ssh username@192.168.1.100
```

**SAMPLE OUTPUT**

**IP CONFIGURATION**

```
inet 192.168.1.100/24
```

**ROUTING TABLE**

```
default via 192.168.1.1 dev ens33
```

**DNS CONFIGURATION**

```
nameserver 8.8.8.8
nameserver 8.8.4.4
```

**OBSERVATIONS**

| Parameter | Status |
|---|---|
| VM Creation | Successful |
| Ubuntu Installation | Successful |
| Static IP Configuration | Successful |
| Subnet Mask Configuration | Successful |
| Gateway Configuration | Successful |
| DNS Configuration | Successful |
| Internet Connectivity | Verified |
| SSH Installation | Successful |
| SSH Service Running | Verified |
| Remote Login | Successful |

**RESULT**

A Virtual Private Server (VPS) was successfully created using Oracle VirtualBox. Ubuntu Server was installed and configured with static IP address, subnet mask, gateway, and DNS settings. SSH service was installed, enabled, and verified for secure remote access.

.

# EXPERIMENT NO: 5

## CLOUD MIGRATION AND BACKUP SETUP USING AMAZON S3

### AIM

To perform cloud migration and backup operations by storing backup files in Amazon S3, automating backup schedules, and restoring data to verify integrity.

### OBJECTIVES

• To understand cloud migration concepts.

• To understand cloud-based backup solutions.

• To create and configure Amazon S3 storage.

• To upload and restore backup data from the cloud.

• To automate backup operations.

• To verify data integrity after restoration.

### THEORY

Cloud Migration is the process of moving data, applications, and workloads from local infrastructure to cloud environments.

Cloud Backup is the process of storing copies of data in cloud storage services to ensure recovery in case of hardware failure, accidental deletion, cyber-attacks, or disasters.Amazon S3 (Simple Storage Service) is a highly scalable object storage service provided by AWS. It offers durability, availability, security, and lifecycle management features suitable for backup and archival purposes.In this experiment, local files are backed up and uploaded to Amazon S3. The backup process is automated using Linux scheduling tools and data is restored to verify successful recovery.

**ARCHITECTURE**

Source Data

↓

Backup Archive Creation

↓

AWS CLI

↓

Amazon S3 Bucket

↓

Cloud Storage

↓

Restore Process

↓

Data Verification

**ALGORITHM**

1. Create AWS account.
2. Create S3 bucket.
3. Install AWS CLI.
4. Configure AWS credentials.
5. Create backup archive.
6. Upload backup to Amazon S3.
7. Verify uploaded files.
8. Restore files from S3.
9. Verify data integrity.
10. Automate backup using cron.

**FLOWCHART**

Start

↓

Create S3 Bucket

↓

Install AWS CLI

↓

Configure AWS Credentials

↓

Create Backup

↓

Upload to S3

↓

Verify Upload

↓

Restore Data

↓

Verify Integrity

↓

Schedule Automated Backup

↓

stop

**PROCEDURE**

**STEP 1: LOGIN TO AWS CONSOLE**

1. Login to AWS Management Console.
2. Navigate to Amazon S3.
3. Open S3 Dashboard.

**STEP 2: CREATE S3 BUCKET**

1. Click **Create Bucket**.
2. Enter bucket name:

```
cloud-backup-lab-bucket
```

3. Select Region.
4. Keep default settings.
5. Click **Create Bucket**.

**STEP 3: INSTALL AWS CLI**

Update system:

```
sudo apt update
```

Install AWS CLI:

```
sudo apt install awscli -y
```

Verify installation:

```
aws --version
```

**STEP 4: CONFIGURE AWS CLI**

```
aws configure
```

Provide:

- AWS Access Key ID
- AWS Secret Access Key
- Region Name
- Output Format

**STEP 5: CREATE SAMPLE DATA**

```
mkdir migration_data

cd migration_data

echo "Cloud Computing Lab Backup" > file1.txt

echo "AWS S3 Backup Experiment" > file2.txt
```

**STEP 6: CREATE BACKUP ARCHIVE**

```
tar -czvf backup.tar.gz migration_data
```

**STEP 7: UPLOAD BACKUP TO AMAZON S3**

```
aws s3 cp backup.tar.gz s3://cloud-backup-lab-bucket/
```

**STEP 8: VERIFY UPLOAD**

```
aws s3 ls s3://cloud-backup-lab-bucket/
```

**Expected Output:**

```
backup.tar.gz
```

**STEP 9: SIMULATE RESTORE OPERATION**

Create restore directory:

```
mkdir restore_data
```

Download backup from S3:

```
aws s3 cp s3://cloud-backup-lab-bucket/backup.tar.gz .
```

Extract backup:

```
tar -xzvf backup.tar.gz -C restore_data
```

**STEP 10: VERIFY RESTORED FILES**

```
ls restore_data/migration_data
```

**Expected Output:**

```
file1.txt
file2.txt
```

**STEP 11: AUTOMATE BACKUP PROCESS**

Create backup script:

```
nano backup_script.sh
```

Script Content:

```
#!/bin/bash

tar -czvf backup.tar.gz migration_data

aws s3 cp backup.tar.gz s3://cloud-backup-lab-bucket/
```

Provide permission:

```
chmod +x backup_script.sh
```

Open cron editor:

```
crontab -e
```

Schedule daily backup:

```
0 10 * * * /home/user/backup_script.sh
```

**COMMANDS USED**

```
aws --version

aws configure

aws s3 ls

aws s3 cp

tar -czvf

tar -xzvf

chmod +x

crontab -e
```

**OBSERVATIONS**

| Parameter | Status |
|---|---|
| S3 Bucket Creation | Successful |
| AWS CLI Installation | Successful |
| AWS Configuration | Successful |
| Backup Creation | Successful |
| Upload to S3 | Successful |
| Restore from S3 | Successful |
| Data Verification | Successful |
| Automation Setup | Successful |

**EXPECTED OUTPUT**

• Amazon S3 bucket created successfully.

• Backup archive uploaded to cloud storage.

• Files restored successfully from S3.

• Data integrity verified.

• Automated backup schedule configured.

**RESULT**

A cloud migration and backup solution was successfully implemented using Amazon S3. Backup files were uploaded to cloud storage, restored successfully, and automated backup scheduling was configured.

# EXPERIMENT NO: 6

## CDN SERVICE SELECTION AND CONFIGURATION USING AMAZON CLOUDFRONT

**AIM**

To select a Content Delivery Network (CDN) provider, configure a CDN distribution, set up DNS records, and deliver web content through the CDN network using Amazon CloudFront.

**OBJECTIVES**

• To understand the concept of Content Delivery Networks (CDN).

• To study CDN providers and their features.

• To configure Amazon CloudFront as a CDN service.

• To configure DNS records for CDN integration.

• To create and manage CDN distributions.

• To verify content delivery through CDN edge locations.

**SOFTWARE REQUIREMENTS**

• AWS Account

• Amazon S3

• Amazon CloudFront

• Domain Name (Optional)

• Web Browser

• Internet Connection

.

**THEORY**

A Content Delivery Network (CDN) is a distributed network of servers that delivers web content from locations closest to users. CDN services reduce latency, improve website performance, enhance availability, and provide security features

Popular CDN providers include:

• Cloudflare

• Amazon CloudFront

• Akamai

• Microsoft Azure CDN

• Google Cloud CDN

Amazon CloudFront is AWS's CDN service that distributes content globally through edge locations. It integrates seamlessly with Amazon S3, EC2, Load Balancers, and custom origins.

DNS (Domain Name System) configuration is essential in CDN deployment. DNS records map domain names to CDN distributions, allowing users to access content through custom domains.

**CDN PROVIDER COMPARISON**

| Provider | Features |
|---|---|
| Cloudflare | Free plan, DNS management, DDoS protection |
| Amazon CloudFront | Global AWS integration, scalable CDN |
| Akamai | Enterprise-grade CDN services |
| Azure CDN | Microsoft ecosystem integration |
| Google Cloud CDN | Integrated with Google Cloud |

**ARCHITECTURE**

User Browser

↓

DNS Resolution

↓

Amazon CloudFront Distribution

↓

Edge Location

↓

Amazon S3 Bucket (Origin)

↓

Content Delivery

**ALGORITHM**

1. Create AWS account.
2. Create S3 bucket.
3. Upload website content.
4. Configure S3 permissions.
5. Create CloudFront distribution.
6. Select S3 bucket as origin.
7. Configure CDN settings.
8. Configure DNS records.
9. Access content through CloudFront URL.
10. Verify CDN delivery.

**FLOWCHART**

Start

↓

Create S3 Bucket

↓

Upload Content

↓

Create CloudFront Distribution

↓

Configure Origin

↓

Configure DNS

↓

Deploy Distribution

↓

Access CDN URL

↓

Verify Content Delivery

↓

Stop

**PROCEDURE**

**STEP 1: LOGIN TO AWS CONSOLE**

1. Open AWS Management Console.
2. Login using AWS credentials.
3. Navigate to Amazon S3.

**STEP 2: CREATE S3 BUCKET**

1. Click **Create Bucket**.
2. Enter bucket name:

```
cdn-lab-bucket
```

3. Select Region.
4. Click **Create Bucket**.

**STEP 3: UPLOAD WEBSITE CONTENT**

Create sample webpage:

```
<html>
<head>
<title>CDN Experiment</title>
</head>
<body>
<h1>Welcome to CloudFront CDN</h1>
</body>
</html>
```

Save as:

```
index.html
```

Upload file to S3 bucket.

**STEP 4: ENABLE STATIC WEBSITE HOSTING**

1. Open S3 Bucket.
2. Navigate to **Properties**.
3. Enable **Static Website Hosting**.
4. Set:

Index Document: `index.html`

5. Save changes.

**STEP 5: CONFIGURE BUCKET PERMISSIONS**

1. Navigate to **Permissions**.
2. Configure bucket policy to allow CloudFront access.
3. Save configuration.

**STEP 6: CREATE CLOUDFRONT DISTRIBUTION**

1. Open Amazon CloudFront Console.
2. Click **Create Distribution**.
3. Under Origin Domain select:

```
cdn-lab-bucket.s3.amazonaws.com
```

4. Configure default settings.
5. Enable caching.
6. Click **Create Distribution**.

**STEP 7: WAIT FOR DISTRIBUTION**

CloudFront creates a unique distribution URL:

Example:

```
d123abcdxyz.cloudfront.net
```

Deployment may take several minutes.

**STEP 8: TEST CDN DISTRIBUTION**

Open browser:

```
https://d123abcdxyz.cloudfront.net
```

Verify webpage loads successfully.

**STEP 9: CONFIGURE CUSTOM DOMAIN (OPTIONAL)**

Suppose domain:

```
www.example.com
```

Open DNS management panel.

**STEP 10: VERIFY DNS RESOLUTION**

Check DNS mapping:

```
nslookup www.example.com
```

OR

```
dig www.example.com
```

Verify CloudFront distribution is returned.

**STEP 11: VERIFY CDN DELIVERY**

Access:

```
https://www.example.com
```

Verify content is served successfully through CloudFront.

**STEP 12: VIEW DISTRIBUTION STATISTICS**

1. Open CloudFront Console.
2. Select Distribution.
3. View:

• Requests

• Data Transfer

• Cache Hit Ratio

• Geographic Distribution

**COMMANDS USED**

```
nslookup www.example.com
```

```
dig www.example.com
```

AWS Services Used:

• Amazon S3

• Amazon CloudFront

**OBSERVATIONS**

| Parameter | Status |
|---|---|
| AWS Account Creation | Successful |
| S3 Bucket Creation | Successful |
| Website Upload | Successful |
| Static Hosting Enabled | Successful |
| CloudFront Distribution Creation | Successful |
| DNS Configuration | Successful |
| CDN URL Access | Successful |
| Content Delivery Verification | Successful |

**EXPECTED OUTPUT**

• Website content uploaded successfully.

• CloudFront distribution created.

• DNS records configured.

• CDN URL accessible.

• Content delivered through CDN edge locations.

• Reduced latency and improved performance.

**RESULT**

A Content Delivery Network was successfully implemented using Amazon CloudFront. Website content was stored in Amazon S3, distributed through CloudFront edge locations, and accessed through CDN URLs and DNS configuration.